MixBytes()

[DEPLOYMENT VERIFICATION REPORT]

Lido Easy Track Factories (CSM 0x02)

PREPARED FOR LIDO

REPORT DATE

October 07, 2026

[CONTENTS]

Table of Contents

1. Introduction 3
    1.1 Disclaimer 3
    1.2 Executive Summary 3
    1.3 Project Overview 4
2. About MixBytes 8

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

This report covers the deployment verification of three Easy Track EVMScript factories deployed for the upcoming Lido Community Staking Module 0x02 (CSM 0x02) mainnet release: `ReportWithdrawalsForSlashedValidators`, `SettleGeneralDelayedPenalty`, and `UpdateStakingModuleShareLimits`. All three are new instances of contracts audited in the "Lido Easy Track Factories (SRv3/CMv2) Security Audit Report"; this engagement does not repeat the code review - it verifies that the newly deployed bytecode is exactly the audited code and that the per-instance constructor configuration is correct and consistent with the CSM 0x02 deployment.

The deployment verification was conducted via on-chain state inspection at block `26128512`, local recompilation of the audited commit with the repository's pinned compiler settings, byte-level bytecode comparison (including cross-comparison against the previously verified June 2026 instances of the same contracts) and cross-checks against the client's deployment artifacts (lidofinance/easy-track#139, lidofinance/staking-modules#899).

The verification vectors below were checked:

- **Bytecode identity vs the audited commit.** Each deployed runtime bytecode was compared byte-for-byte against a local compile of the audit commit `b5d74557448a7f65ea74abdc09ff954bd113c8c3` and against the live June 2026 deployments of the same contracts. Every executable byte matches; the only differences are the constructor-immutable slots and the 32-byte IPFS metadata hash.
- **Constructor arguments and immutables.** Deployment-transaction calldata, the artifact manifests, and the on-chain immutables read back from each contract were cross-checked for equality, including `trustedCaller`, `module`, `accounting`, `stakingRouter`, `stakingModuleId`, the per-motion delta caps, and the instance name.
- **Access control.** The `trustedCaller` multisig (CSM committee Safe) was re-verified: threshold, owner set, Safe version, and singleton.
- **Registry and governance state.** Easy Track factory-registry status and the StakingRouter module registry were read on-chain to establish the deployment's position in the rollout sequence.

Out of scope of this verification are the CSM 0x02 module contracts themselves (`CSModule`, `Accounting`, `ParametersRegistry`, `FeeDistributor`, `Verifier`, `FeeOracle`, `HashConsensus`, `ValidatorStrikes`, `Ejector`, `ExitPenalties`, `PermissionlessGate` and the shared libraries from `lidofinance/staking-modules` at `4d3de6658499e1c1774951780a97d7ae25ca18b8`). No code review was performed in this engagement.

1.3 Project Overview

Summary

CLIENT	Lido
CATEGORY	Liquid Staking
PROJECT	Easy Track Factories (CSM 0x02) - Deployment Verification
TYPE	Solidity
PLATFORM	EVM
TIMELINE	05.10.2026 - 06.10.2026

Scope of Audit

`contracts/EVMScriptFactories/ReportWithdrawalsForSlashedValidators.sol`

`contracts/EVMScriptFactories/SettleGeneralDelayedPenalty.sol`

`contracts/EVMScriptFactories/UpdateStakingModuleShareLimits.sol`

Versions Log

DATE	COMMIT HASH	NOTE
27.05.2026	`9fedee6aede0898d2c924e4b99ce688a352dd11c`	Initial Commit (previous audit)
16.06.2026	`b5d74557448a7f65ea74abdc09ff954bd113c8c3`	Commit for Re-audit (previous audit)
05.10.2026	`b5d74557448a7f65ea74abdc09ff954bd113c8c3`	Deployment Verification Commit

The scope of this engagement is the verification of the three deployments listed below against the audited commit of the previous report "Lido Easy Track Factories (SRv3/CMv2) Security Audit Report". No new code review was performed; the three files above are unchanged between the initial audit commit `9fedee6aede0898d2c924e4b99ce688a352dd11c`, the re-audit commit `b5d74557448a7f65ea74abdc09ff954bd113c8c3`, and the CSM 0x02 deployment branch `feat/csm0x02-mainnet-factories` (lidofinance/easy-track#139), as is their full in-repository import closure (`TrustedCaller`, `EVMScriptCreator`, `IEVMScriptFactory`, `IBaseModule`, `IStakingRouter`).

[1 INTRODUCTION]

Mainnet Deployments

FILE	ADDRESS	BLOCKCHAIN
ReportWithdrawalsForSlashedValidators.sol (CSM0x02)	0x8D74020d8EACCdFf0366dAAFfb96e6c98CDFc112	Ethereum
SettleGeneralDelayedPenalty.sol (CSM0x02)	0x0B676AdEABcf4A696187cfAb90290Aa3ac51aFA2	Ethereum
UpdateStakingModuleShareLimits.sol (CSM0x02)	0x5b0De22E65C068430f6e769754D51133775408cc	Ethereum

Deployment transactions. All three contracts were deployed by the same EOA `0x1132510B0fBB75389359dFC5630C252CC950D408` in contract-creation transactions that match the hashes published in the deployment artifacts of lidofinance/easy-track#139 (`deployed-sm-mainnet.json` / `deployed-sr-mainnet.json`):

CONTRACT	ADDRESS	TX HASH	BLOCK
ReportWithdrawalsForSlashedValidators (CSM0x02)	0x8D74020d8EACCdFf0366dAAFfb96e6c98CDFc112	0x5bbbad55bab3d8334b89d2404d571f9b60206d7c9991cf9fe5ed2e625cc62511	26104953
SettleGeneralDelayedPenalty (CSM0x02)	0x0B676AdEABcf4A696187cfAb90290Aa3ac51aFA2	0x50c316af935e58eb45a3a10798f7bc6f44cda95ddec7eed882ee3b76c76198c0	26104965
UpdateStakingModuleShareLimits (CSM0x02)	0x5b0De22E65C068430f6e769754D51133775408cc	0x52a2eac1bb15f4e585e5307467d54c3342c35c1ac2a7d58737c1698492514652	26104984

Bytecode identity. Each contract was compiled locally from the audit commit `b5d74557448a7f65ea74abdc09ff954bd113c8c3` with the repository's pinned settings (solc `0.8.6`, EVM version `berlin`, optimizer disabled - `brownie-config.yaml`; the factories pull in no external libraries). For every deployment the on-chain runtime code (`eth_getCode`) equals the compiled code. The same comparison against the live June 2026 audited instances of the same contracts (`ReportWithdrawalsForSlashedValidators (CSM)` 0xE330516a03bDdEBA4209b5591112f1aa3dd90F0A, `SettleGeneralDelayedPenalty (CSM)` 0xB71755bE764abB4Ce26cb4dADf056Be57fB8880F, `UpdateStakingModuleShareLimits (CSM)` 0xde3e46E3129fA4e4e3f66c9024B0A3Ad509b27a1) is likewise byte-identical outside immutables and the metadata hash.

All contracts are non-proxy with no initializers, owner roles, or upgrade paths; their entire configuration is the constructor-immutable set verified below.

Immutable configuration. Every value matches the constructor arguments published in lidofinance/easy-track#139, which also match the constructor-argument encoding in the deployment-transaction calldata:

CONTRACT AND IMMUTABLE	EXPECTED	ON-CHAIN READ	RESULT
`ReportWithdrawalsForSlashedValidators: trusted Caller`	0xC52fC3081123073078698F1EAc2f1Dc7Bd71880f	0xC52fC3081123073078698F1EAc2f1Dc7Bd71880f	match
`ReportWithdrawalsForSlashedValidators: name`	"CSM0x02"	"CSM0x02"	match
`ReportWithdrawalsForSlashedValidators: module`	0x792Cd25e4aE3578375031FB55e048E163A804F7B	0x792Cd25e4aE3578375031FB55e048E163A804F7B	match
`SettleGeneralDelayedPenalty: trustedCaller`	0xC52fC3081123073078698F1EAc2f1Dc7Bd71880f	0xC52fC3081123073078698F1EAc2f1Dc7Bd71880f	match
`SettleGeneralDelayedPenalty: name`	"CSM0x02"	"CSM0x02"	match
`SettleGeneralDelayedPenalty: module`	0x792Cd25e4aE3578375031FB55e048E163A804F7B	0x792Cd25e4aE3578375031FB55e048E163A804F7B	match
`SettleGeneralDelayedPenalty: accounting`	0x3696dDd942A9e156F5D4728505D1b9a32dCef900	0x3696dDd942A9e156F5D4728505D1b9a32dCef900	match
`UpdateStakingModuleShareLimits: trustedCaller`	0xC52fC3081123073078698F1EAc2f1Dc7Bd71880f	0xC52fC3081123073078698F1EAc2f1Dc7Bd71880f	match
`UpdateStakingModuleShareLimits: name`	"CSM0x02"	"CSM0x02"	match
`UpdateStakingModuleShareLimits: stakingRouter`	0xFdDf38947aFB03C621C71b06C9C70bce73f12999	0xFdDf38947aFB03C621C71b06C9C70bce73f12999	match
`UpdateStakingModuleShareLimits: stakingModule Id`	5	5	match
`UpdateStakingModuleShareLimits: maxStakeShare LimitIncrease / maxStakeShareLimitDecrease`	50 / 50 bp	50 / 50 bp	match
`UpdateStakingModuleShareLimits: maxPriority ExitShareThresholdIncrease / maxPriority ExitShareThresholdDecrease`	60 / 60 bp	60 / 60 bp	match

Integration wiring (present at block `26128512`). The bindings resolve consistently:

- `CSModule(0x792Cd25e4aE3578375031FB55e048E163A804F7B).ACCOUNTING()` returns `0x3696dDd942A9e156F5D4728505D1b9a32dCef900`, which equals the `accounting` immutable of the `SettleGeneralDelayedPenalty` instance; the module is fresh (`getNodeOperatorsCount() == 0`).
- `StakingRouter(0xFdDf38947aFB03C621C71b06C9C70bce73f12999).getStakingModuleIds() == [1, 2, 3, 4]`; `getStakingModule(5)` reverts with `StakingModuleUnregistered()`. Module 5 is the registration slot the pending governance vote will create for CSM 0x02; until then `UpdateStakingModuleShareLimits.validateParams` reverts, so no share-limit motions can be created through the new factory.
- The selectors the factories emit resolve against the live targets: `reportSlashedWithdrawnValidators((uint256,uint256,uint256,uint256,bool)[])` and `settleGeneralDelayedPenalty(uint256[],uint256[])` are declared in the deployed module's

`IBaseModule`, and the self-referential `validateParams((uint16,uint16,uint16,uint16))` plus `updateModuleShares(uint256,uint16,uint16)` match the StakingRouter's interface.

- The `trustedCaller` - the CSM committee Safe 0xC52fC3081123073078698F1EAc2f1Dc7Bd71880f - is unchanged since the June 2026 audit: Safe `v1.3.0` on the canonical singleton `0xd9Db270c1B5E3Bd161E8c8503c55cEABeE709552`, threshold `4`-of-`6` with the same owner set.

Easy Track registry state and expected registration.
`EasyTrack(0xF0211b7660680B49De1A7E9f25C65660F0a13Fea).isEVMScriptFactory(...)` returns `false` for each of the three new instances, while the same call returns `true` for the audited CSM instances - the new factories are not yet registered in the Easy Track registry, and their registration is part of the pending DAO vote. The module-side enactment wiring, however, is already live: the CSM 0x02 `CSModule` (`0x792Cd25e4aE3578375031FB55e048E163A804F7B`) granted `SETTLE_GENERAL_DELAYED_PENALTY_ROLE` and `REPORT_SLASHED_WITHDRAWN_VALIDATORS_ROLE` to the Easy Track `EVMScriptExecutor` (`0xFE5986E06210aC1eCC1aDCafc0cc7f8D63B3F977`) during the deployment. Registration of the factories is therefore the only missing step before committee motions can be created and enacted.

[2 ABOUT MIXBYTES]

Built on Trust

MixBytes is a blockchain security firm specializing in the analysis of decentralized protocols and smart contract systems. The company helps Web3 teams build resilient protocol architecture, smart contract logic, and economic mechanisms through a combination of AI-assisted analysis and senior human expertise.

Rather than focusing solely on one-time audits before deployment, MixBytes works with protocols across their entire lifecycle - from early architecture design to production audits and ongoing protocol evolution. Our team consists of experienced security researchers, engineers, and protocol analysts with deep expertise in DeFi systems, adversarial protocol analysis, and smart contract security.

Over the years, MixBytes has worked with many of the most widely used protocols in the ecosystem, including Lido, Aave, Curve, 1inch, OKX, Mantle, Fluid, Gearbox, Resolv, and others. To enhance analysis efficiency and coverage, MixBytes also develops and uses internal AI-assisted tooling that helps surface potential risk signals during development and code review.

SECURED TVL	AUDITS DELIVERED	WEB3 PROTOCOLS
$50B+	300+	80+

Protocol Security Lifecycle

MixBytes supports protocols across the full lifecycle of their development and operation.

Web-3 Design Review

Independent assessment of protocol architecture and economic design before critical decisions are embedded in production code.

AI Tooling

AI-assisted analysis integrated into development workflows to surface potential risk signals during protocol development.

Smart Contract Audit

Comprehensive manual verification of smart contract logic and protocol invariants before production deployment.

Security Support

Continuous expert support for protocol upgrades, integrations, governance changes, and evolving attack surfaces after launch.

CONTACTS

hello@mixbytes.io